

Pointers

CSC 240 Data Structures Fall 2025

Table of Contents

- [1. Objectives](#)
- [2. Getting the Address of a Variable](#)
- [3. PRACTICE Using the address-of operator](#)
- [4. Pointer Variables](#)
- [5. Reference variables](#)
- [6. Pointers vs. references and reason for pointers](#)
- [7. Creating and using pointer variables](#)
- [8. PRACTICE Store integer address in a pointer](#)
- [9. C++urious George](#)
- [10. Indirect access to variables](#)
- [11. Re-pointing pointers](#)
- [12. PRACTICE Working with pointers](#)
- [13. C++urious George](#)
- [14. PRACTICE Review questions](#)

1. Objectives

We don't assume any knowledge of pointers but if you do have any, it will be useful to review the concept and the practice of pointers.

1. Understand what a pointer is and how it differs from a reference.
2. Use pointers with arrays and functions.
3. Perform basic pointer arithmetic.
4. Dynamically allocate and deallocate memory using raw pointers.
5. Introduce the concept of smart pointers and avoid memory leaks.

All topics:

- [] Getting the Address of a Variable
- [] Pointer and Reference Variables
- [] Relationship between Arrays and Pointers
- [] Pointer Arithmetic
- [] Initializing Pointers
- [] Pointers as Function Parameters
- [] Dynamic Memory Allocation

2. Getting the Address of a Variable

- The address-of operator & returns the memory address of a variable.
- Where exactly is this "memory address" on the computer?

The memory address of most local variables is in the RAM (Random Access Memory) portion of the computer, somewhere in the **stack** segment. Dynamically allocated memory

that can shrink or grow at runtime (via `new` or `malloc`) is in the **heap** segment.

- When the variable is undefined, you get any old value:

```
double amount; // double precision floating-point variable
cout << "Address: " << &amount // address-of variable
    << ", value = " << amount // value of variable
    << ", Size = " << sizeof(amount) << " bytes." << endl;
```

Address: 0x7ffd5cf78a30, value = 6.27853e-310, Size = 8 bytes.

- Why are the addresses always different?

Because of ASLR (Address Space Layout Randomization): Done for security reasons - much harder to inject code in changing memory.

- The following fragment declares three variables.

```
char letter; // 1 byte ( 8 bits)
short number; // 2 bytes (16 bits)
float amount; // 4 bytes (32 bits)
```

- In the computer memory, they may be arranged like this:

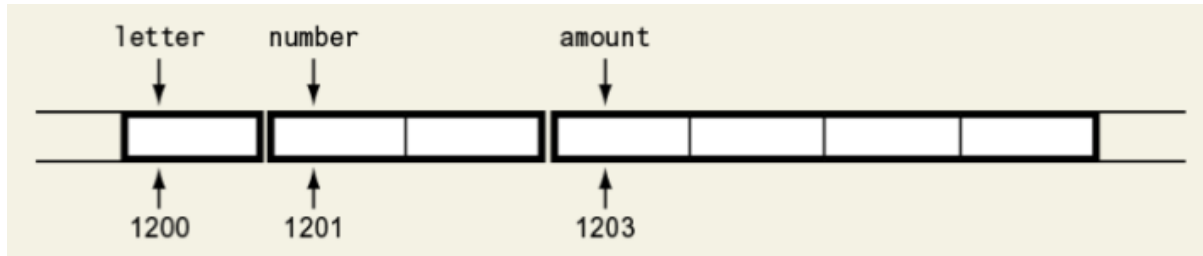


Figure 1: Variables and their addresses (Gaddis fig 9-1).

- The address itself is represented in hexadecimal (base 16) format. On a 64-bit computer system, the length of an address is typically 64 bits or 8 bytes.
- How many addresses can be represented on a 64-bit computer system?

On a 64-bit computer system, every bit can have 2 values (0 or 1), which means that there are 2^{64} (18 quintillion) unique addresses.

Incidentally, the reason why No Man's Sky can generate about 18 quintillion planets is because their universe uses a 64-bit generation seed leading to that many possible planet addresses.

- Is each byte in the computer's memory assigned to a unique address? If so, how would we know this?

Yes, each byte in a computer's memory is assigned to a unique address.

- Demonstration:

```
char a = 'A', b = 'B'; // two variables defined consecutively
printf("%p\n", (void*)&a); // using a 'generic' pointer `void*`
printf("%p\n", (void*)&b); // addresses 1 byte apart
```

```
0x7fff838c46b6
0x7fff838c46b7
```

3. **PRACTICE** Using the address-of operator

- Write a C++ program that
 1. Declares an integer variable x and assigns it the value 25.
 2. Prints the address of x, &x.
 3. Prints the size of x in bytes, sizeof(x)
 4. Prints the value stored in x.

- Sample output:

```
The address of x is 0x7ffc1970c684
The size of x is 4
The value stored in x is 25
```

- Solution:

```
// This program uses the & operator to determine the address of a
// variable and the sizeof operator to determine its size.

#include <iostream>
using namespace std;

int main()
{
    int x = 25;

    cout << "The address of x is "      << &x      << endl
         << "The size of x is "        << sizeof(x) << endl
         << "The value stored in x is " << x        << endl;
    return 0;
}
```

```
The address of x is 0x7ffd78596ae4
The size of x is 4
The value stored in x is 25
```

```
The address of x is 0x7ffe0929bf24
The size of x is 4
The value stored in x is 25
```

4. Pointer Variables

- A pointer variable is a variable that holds a memory address. When it is used to point at data, its type has to coincide with the type of the data: An integer pointer can point at an integer, etc.
- When an array is passed to a function, it "decays" to a pointer to the first array element.
- What happens in the following code example with regard to variable addresses?

```
#include <iostream>
using namespace std;

void showValues(int [], int);

int main()
{
    const int SIZE = 5;
    int numbers[SIZE] = {1,2,3,4,5};
    showValues(numbers, SIZE);
    return 0;
}

void showValues(int values[], int size)
{
    for (int count = 0; count < size; count++)
        cout << values[count] << " ";
}
```

1 2 3 4 5

- Explanation:
 1. In main, the name of the array numbers and its SIZE are passed to the function showValues.
 2. In the function, the values parameter receives the **address** of the array numbers. It points to numbers[0].
 3. Inside the function, anything done to the values parameter is done to the numbers array.

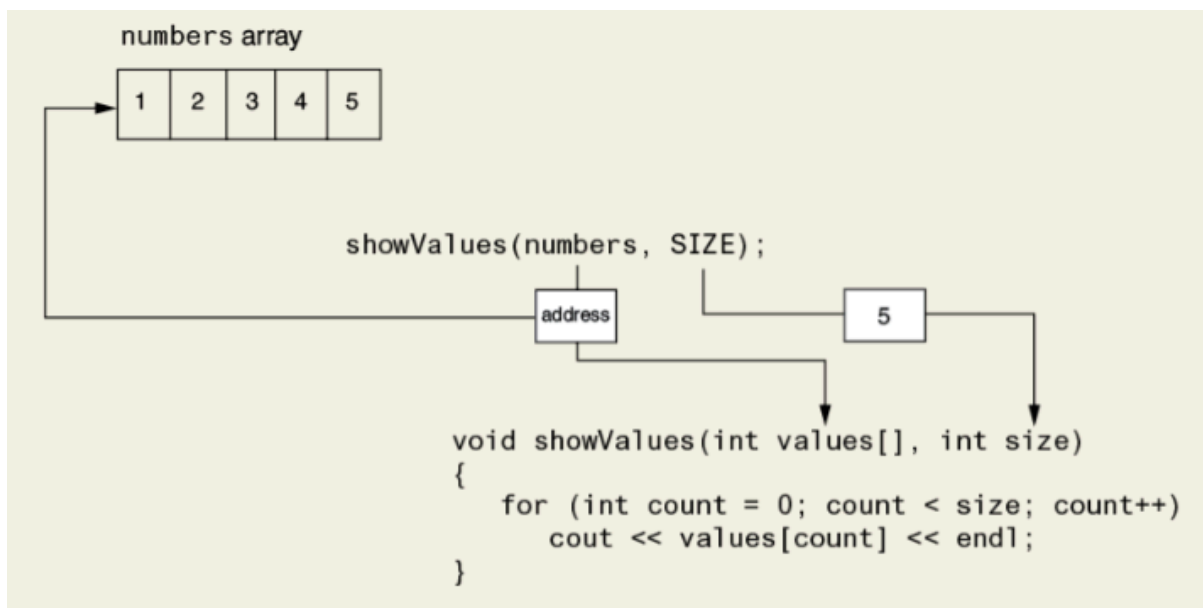


Figure 2: Passing an array to a function (fig. 9-2, Gaddis)

5. Reference variables

- A reference variable acts as an alias for another variable. Anything you do to the reference variable is done to the variable it references.
- What happens in the following code example with regard to variable references?

```
#include <iostream>
using namespace std;

void getOrder(int&);

int main()
{
    int jellyDonuts;
    getOrder(jellyDonuts);

    return 0;
}

void getOrder(int& donuts)
{
    cout << "How many donuts do you want? ";
    cin >> donuts;
    cout << "\nOkay, " << donuts << " it is.\n\n";
}
```

```
How many donuts do you want?
Okay, 4 it is.
```

- Explanation:
 1. In the function getOrder, the donuts parameter is a **reference** variable.
 2. The reference variable receives the **address** of the jellyDonuts variable (the argument of the getOrder function in main).
 3. donuts works like a pointer because it points to the jellyDonuts variable.

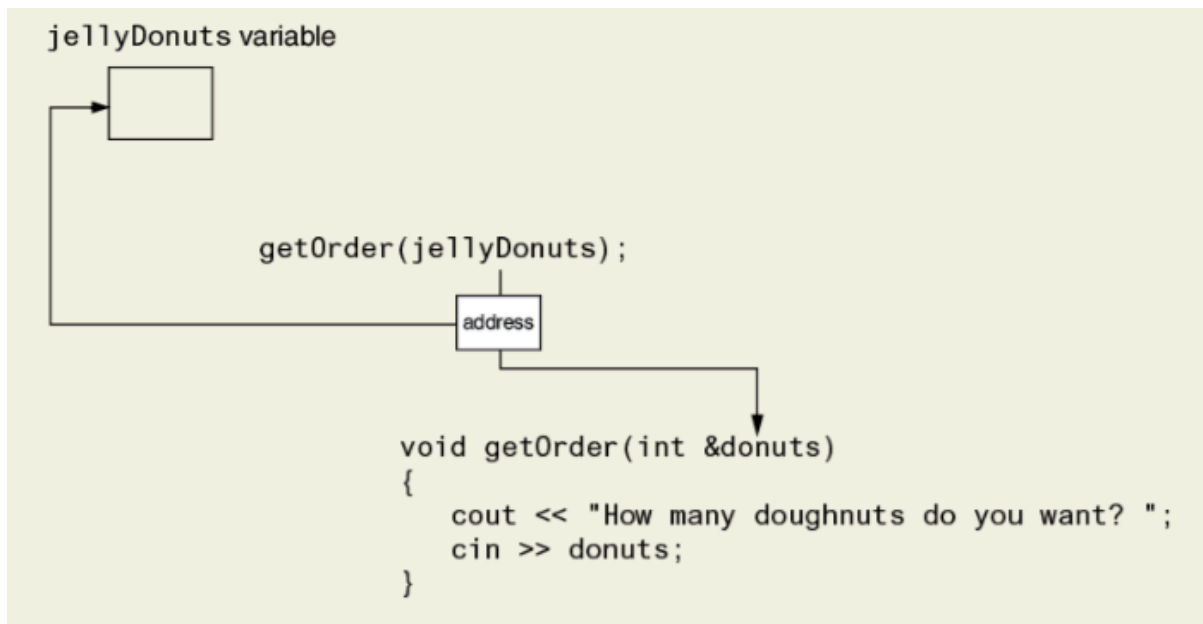


Figure 3: Passing an argument by reference (fig. 9-3, Gaddis)

- Compile and run the code above on the shell.

```
cd ../src
g++ donuts.cpp -o donuts
echo "4" | ./donuts
```

```
How many donuts do you want?
Okay, 4 it is.
```

- Create input file ../data/donuts.txt

```
cd ../data
echo "4" > donuts.txt
cat donuts.txt
```

```
4
```

6. Pointers vs. references and reason for pointers

- Pointer variables are similar to reference variables but they require more work from you.
- To make a pointer variable reference another item in memory, you have to write code to **fetch** the memory address of that item and **assign** it to the pointer variable (using the address-of operator &).
- When you use a pointer to **store** a value in the memory location that the pointer references, you have to specify that it should be stored at a location rather than in the pointer variable itself (using the indirection or dereference operator *).

- Programmers use reference variables to make changes permanent between calling routines and functions. But why use pointers?
 1. Pointers are necessary for **dynamic memory allocation**: whenever you need to work with an unknown amount of data.
 2. Pointers are useful in **algorithms** that manipulate arrays and especially strings (text).
 3. In object-oriented programming (OOP), pointers are useful for creating and working with objects and for **sharing access**.
- Example: Walking through a string using a pointer

```
#include <iostream>

int main() {
    const char* greeting = "Hello, world!"; // Declare a C-style string
                                           // as constant (literal)
                                           // character pointer.
    std::cout << greeting << std::endl;    // Print the whole string.

    // The pointer points at greeting[0].
    std::cout << "First letter: " << *greeting << std::endl;    // 'H'
    // Moving forward, the pointer points at greeting[1]
    std::cout << "Next letter: " << *(greeting + 1) << std::endl; // 'e'

    // Traverse the C-style string with a pointer
    const char* p = greeting;
    while (*p != '\0') {
        std::cout << *p;
        ++p;
    }
    std::cout << std::endl;
    return 0;
}
```

```
Hello, world!
First letter: H
Next letter: e
Hello, world!
```

7. Creating and using pointer variables

- What does `int* ptr;` mean?
 1. `ptr` is a pointer variable that points to an integer variable.
 2. `ptr` can hold the value of a computer memory address.
 3. Currently, `ptr` is unassigned (it can point towards any address).
- What's the difference between `int* ptr` and `int *ptr`?

There is no difference between `int* ptr` and `int *ptr`.

`int* ptr` emphasizes that the data type of `ptr` is not `int` but pointer-to-`int`, and distinguishes the use of `*` from its use as indirection/dereference operator.

- **Rule:** Always initialize a pointer variable with the `nullptr` - otherwise you're changing a memory address somewhere.

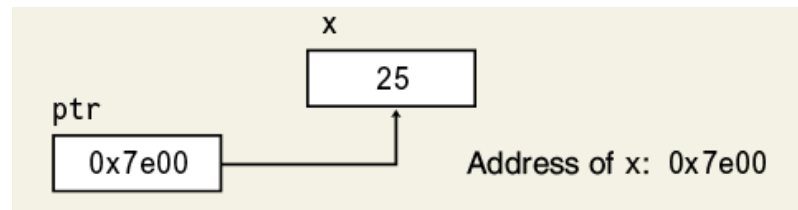
```
int* foo = nullptr; // C++11
int* bar = NULL;    // C or older C++

printf("%p %p\n", foo, bar); // (nil) is NULL in Emacs Lisp
cout << foo << " " << bar;  // prints 0
```

```
(nil) (nil)
0 0
```

8. PRACTICE Store integer address in a pointer

- The following program defines an integer variable `x`, assigns a value to it, defines a pointer `ptr` and assigns the null pointer to it. Then the pointer is made to point at the variable address and both the value and the address are printed.



- Task:** Open the starter code in OneCompiler.com, comment it where indicated, and complete the `cout` statements.
- Starter code:

```
// Store integer variable address in a pointer.
#include <iostream>
using namespace std;

int main()
{
    int x = 25;           // ...
    int *ptr = nullptr;   // ...
    ptr = &x;             // ...

    cout << "The value in x is " << ____ << endl;
    cout << "The address of x is " << ____ << endl;

    return 0;
}
```

```
The value in x is 25
The address of x is 0x7fff60a94f4c
```

- Solution (working example with comments)

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int x = 25;           // Assign value to integer variable.
    int *ptr = nullptr;   // Pointer points nowhere in particular.
    ptr = &x;             // Store address of variable in pointer

    cout << "The value in x is " << x << endl;
    cout << "The address of x is " << ptr << endl;

    return 0;
}
```

```
The value in x is 25
The address of x is 0x7ffd8b5a35ac
```

9. C++urious George



- Can one assign a specific address directly to a pointer?

Yes, one can, but it is dangerous and almost never recommended. You can assign a specific address to a pointer using a cast, like `int* p = (int*)0x7ffee4a2b4fc;`. However, if you access an invalid or restricted memory location, your program may crash or cause undefined behavior. Safe pointer use relies on assigning the address of a known variable or using dynamic memory allocation.

```
int* p = (int*)0x7ffee4a2b4fc;
cout << p; // *p will give segmentation fault
```

```
0x7ffee4a2b4fc
```

- What does the address `0x7e00` tell us about the memory layout of the program (e.g., stack location, variable storage, or memory limits), and how does it relate to the system's overall memory architecture?

The address 0x7e00 shows that the variable or data it refers to is located at physical memory address 32,768 (in decimal). This is a relatively low address - you'd never see such a low address.

However, the address alone does not reveal whether the system is 16-, 32-, or 64-bit—it simply indicates that this specific value fits within all of those architectures.

```
echo "ibase=16; 7E00;" | bc # 7 * 16^3 + 14 * 16^2 = 7 * 4096 + 14 * 256
```

```
32256
```

10. Indirect access to variables

- Pointers allow you to indirectly access and modify the variable being pointed to using the **indirection** operator *
- Example program:
 1. Assign the address of a variable to a pointer.
 2. Print the value of the variable using the dereferenced pointer.
 3. Assign a value to the location pointed to by the pointer.
 4. Print the value of the variable using the dereferenced pointer.

```
// This program demonstrates the use of the indirection operator.
#include <iostream>
using namespace std;

int main()
{
    int x = 42;           // Integer variable
    int *ptr = nullptr;   // Pointer variable that can point to an int
    ptr = &x;             // Store the address of x in the pointer

    // Use x and ptr to display the VALUE in x
    cout << "Value of x (direct): " << x << endl
         << "Value of x (indirect via pointer): " << *ptr << endl;

    // Assign 99 to the location pointed to by ptr.
    // This will assign 99 to x.
    (*ptr) = 99;

    // Use x and ptr to display the VALUE in x
    cout << "Value of x (direct): " << x << endl
         << "Value of x (indirect via pointer): " << *ptr << endl;

    return 0;
}
```

```
Value of x (direct):          42
Value of x (indirect via pointer): 42
Value of x (direct):          99
Value of x (indirect via pointer): 99
```

- Explanation:

```
[ptr] ---> [x: 42] // ptr is assigned &x
[ptr] ---*---> 42 // *ptr holds 42
[ptr] ---*---> 99 // *ptr is assigned 99
[ptr] ---> [x: 99] // x holds 99
```

11. Re-pointing pointers

- Since pointers hold memory addresses, they can be re-pointed to other addresses. That happens when you initialize a pointer with the `nullptr` and then assign the address `&x` of an existing variable `x`.
- The following code uses one pointer `ptr` to change the values in three different variables:

```
// Define two integer variables and one integer pointer.
int x = 25, y = 50;
int *ptr = nullptr;

// Display the contents of the integer variables.
cout << "x = " << x << ", y = " << y << endl;

// Use the pointer to manipulate the variables.

ptr = &x;           // Store address of x in pointer
                    // Pointer points at x
(*ptr) += 100;      // Add 100 to the value in x

ptr = &y;           // Store address of y in pointer
                    // Pointer points at y
(*ptr) += 100;      // Add 100 to the value in y

// Display the contents of the integer variables.
cout << "x = " << x << ", y = " << y << endl;
```

```
x = 25, y = 50
x = 125, y = 150
```

12. **PRACTICE** Working with pointers

- Complete the starter code in:
 1. Initialize the pointer with the `nullptr`.
 2. Assign the address of the variable to the pointer.
 3. Change the value of the variable using a pointer.
 4. Display the contents of the variable using a pointer.
- Sample output:

```
The number's value is 99.
```

- Starter code:

```
// This program demonstrates working with pointers.
#include <iostream>
using namespace std;

int main()
{
    // Assign 42 to the integer `number`.

    // Assign the null pointer to an integer pointer `ptr`.

    // Assign the address of number to the pointer.

    // Change value of number to 99 using the pointer.

    // Display the contents of the variable number using a pointer.

    return 0;
}
```

The number's value is 99.

- Solution code:

```
// This program demonstrates working with pointers.
#include <iostream>
using namespace std;

int main()
{
    int number = 42; // Assign 42 to number.

    int *ptr = nullptr; // Assign null pointer to integer pointer.

    ptr = &number;    // Assign address of number to pointer.

    (*ptr) = 99;      // Change value of number to 99 using the pointer.

    // Display the contents of the variable number using a pointer.
    cout << "The number's value is " << *ptr << ".\n";

    return 0;
}
```

The number's value is 99.

13. C++urious George



- What happens if you assign the address of a non-existing variable to a pointer?

```
int *ptr = num; // num was not declared in this scope - compile error
```

- A pointer stores the value of an address to a variable. When it is re-pointed so that it stores the value of another variable, what happens to the address of the first variable?

Nothing happens to the address of the first variable. It still exists in memory. Only the pointer has changed to store a new address.

Unless there are other pointers pointing to the first variable, its address remains unused but intact. This is different from **dynamic memory**, where losing the only pointer to allocated memory causes a memory leak.

- Where exactly is the `nullptr` pointing at?

`nullptr` is a special constant in C++ that represents a pointer that does not point to any valid memory location. Internally, it is represented as the address 0 (or a special null representation), but you should not assume it maps to a real memory region.

Accessing or dereferencing a `nullptr` results in undefined behavior, typically a segmentation fault.

- What happens if you try to access or dereference a `nullptr`?

Accessing a `nullptr`'s value is undefined (may return zero). Dereferencing a `nullptr` results in undefined behavior, typically a segmentation fault.

```
int *ptr = nullptr;
//cout << *ptr;
```

- If `ptr` is a pointer variable that holds the value of an address, and `*ptr` is the value of that variable, what is `**ptr` and what is `&*ptr` and are these allowed? Test your assumptions.

```
int foo = 42; // integer variable
int *ptr = &foo; // pointer-to-int (stores address of int)
int **ptr2 = &ptr; // pointer-to-pointer-to-int (stores address of
                  // pointer-to-int)
cout << ptr << " " << *ptr << " " << &*ptr << endl;
cout << *ptr2 << " " << **ptr2;
```

```
0x7ffffd75061e4 42 0x7ffffd75061e4
0x7ffffd75061e4 42
```

14. PRACTICE Review questions

1. Write a statement that defines a pointer variable named `fltPtr`. The variable should be a pointer to a float. Be sure to properly initialize the variable.

```
float *fltPtr = nullptr;
```

2. Define an integer variable `foo` and a pointer-to-int variable `ptr`. Write a statement that assigns the address of the `foo` variable to the `ptr` variable.

```
int foo; // Does not have a defined value.
int *ptr=nullptr; // Points towards 0 or NULL.
ptr = &foo; // Store address of foo in ptr.
```

3. Does `foo` have to have a value for this to work?

No, `foo` does not have to have a value for this to work. `ptr` holds the value of the **address** of `foo`. The value `*ptr` is undefined.

4. Define `ptr` as a pointer variable that points to another variable `foo`. Write a `cout` statement that displays the value that `ptr` points to.

```
int foo;
int *ptr = &foo; // the asterisk indicates that ptr is a pointer variable
cout << *ptr; // the asterisk dereferences the pointer to access the
              // value of the variable the pointer points to
```

```
29726
```

5. List three uses of the `*` (asterisk) symbol in C++.

1. Multiplication symbol.
2. Pointer definition symbol.
3. Indirection operator (dereferencing symbol).

6. What is the output of the following code?

```
int x=50, y=60, z=70;
int *ptr = nullptr;
```

```
cout << x << " " << y << " " << z << endl; // 50 60 70

ptr = &x;
(*ptr) *= 10; // 500
ptr = &y;
(*ptr) *= 5; // 300
ptr = &z;
(*ptr) *= 2; // 140

cout << x << " " << y << " " << z << endl; // 500 300 140
```

```
50 60 70
500 300 140
```

Author: Marcus Birkenkrahe

Created: 2025-09-30 Tue 08:54